Deliver AI with Less Compute

A budget-aware controller for "ask-your-data" analytics: route each question to the right model, send only the schema it needs, verify only where a mistake is likely.

−68 % compute per correct answer on 150 unseen tasks, at −0.6 pp accuracy.

Ahmed Sharahili · solo · 30–31 Oct 2026 · github.com/Ahmednsh/humain-less-compute

Every question gets the biggest model and the whole context

1

Tokens the task never reads

Full 12-table schema, a long glossary and nine examples on every call: 3 461 prompt tokens for "how many cities are in Asir".

2

Failed runs still burn compute

Repairs, retries and verification calls cost money whether or not the answer ends up right, so the metric that matters is cost per successful task.

3

Cheap only counts if it is right and on time

Any saving has to be judged under an accuracy floor and a latency floor fixed in advance.

3 461

prompt tokens per question in the baseline

~1 000

after pruning, same answers

One controller in front of two model tiers

Small: Llama 3.3 70B · Large: Claude Sonnet 5.5 · both via OpenRouter · ≈20× price gap · the small tier was chosen by a measured bake-off, not by reputation

1

Route

A zero-token lexical difficulty score sends easy questions to the small model and hard ones straight to the large one.

2

Prune

Only the tables the question mentions plus the join path between them, the glossary lines it needs and at most two worked examples.

3

Verify where it pays

Free checks on every answer. A large-model second opinion only for risky small-tier answers: low confidence, complex SQL or money.

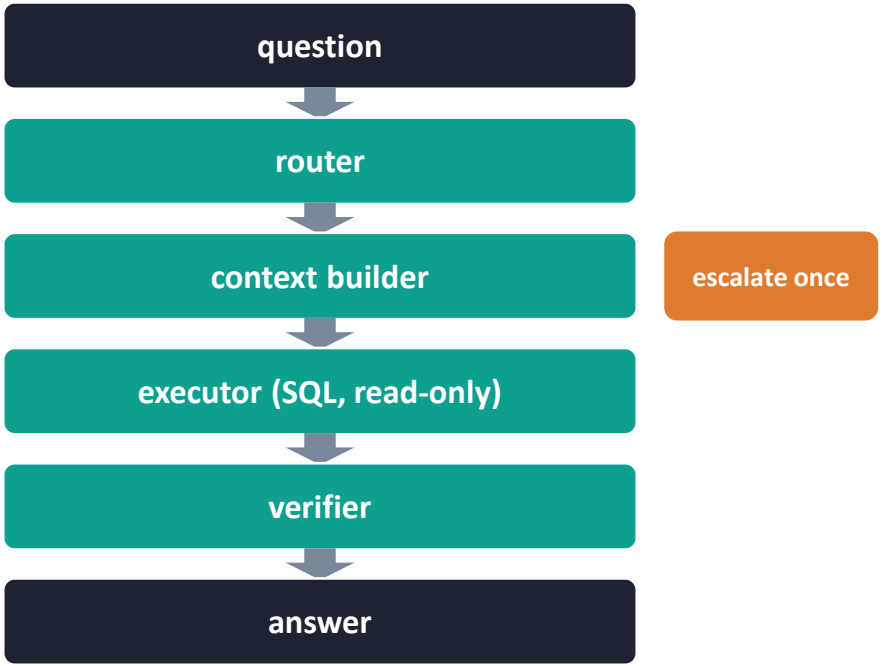
4

Escalate once

If checks fail, the large model takes over with the same pruned context. The failed attempt is still charged.

It is an agent in the challenge's sense: it observes outcomes (errors, empty results, verdicts) and decides where the next token goes.

Technical approach



Data

Deterministic 12-table SQLite company (regions, cities, customers, orders, items, payments, shipments, returns, employees, tickets), 6 000 orders from one seed.

Tasks

33 question templates in three difficulty tiers, English and Arabic phrasings, gold SQL executed on the same DB: an exact, free oracle, hand-checked on a fixture.

Agent

TypeScript on Node 26, node:sqlite in a killable child process with a deadline, write guard, JSON output parsed with zod, repair loop.

Harness

Every call logged (tokens, USD from OpenRouter, latency); resumable JSONL runs; one command rebuilds the report from the logs.

Floors fixed first, every component gets a row

The ablation ladder

P0	large model always (baseline)
S	small model only
P1	+ router
P2	+ context pruning
P3	+ selective verification = full system
V	verify always (contrast)

Floors, fixed before the first optimisation run

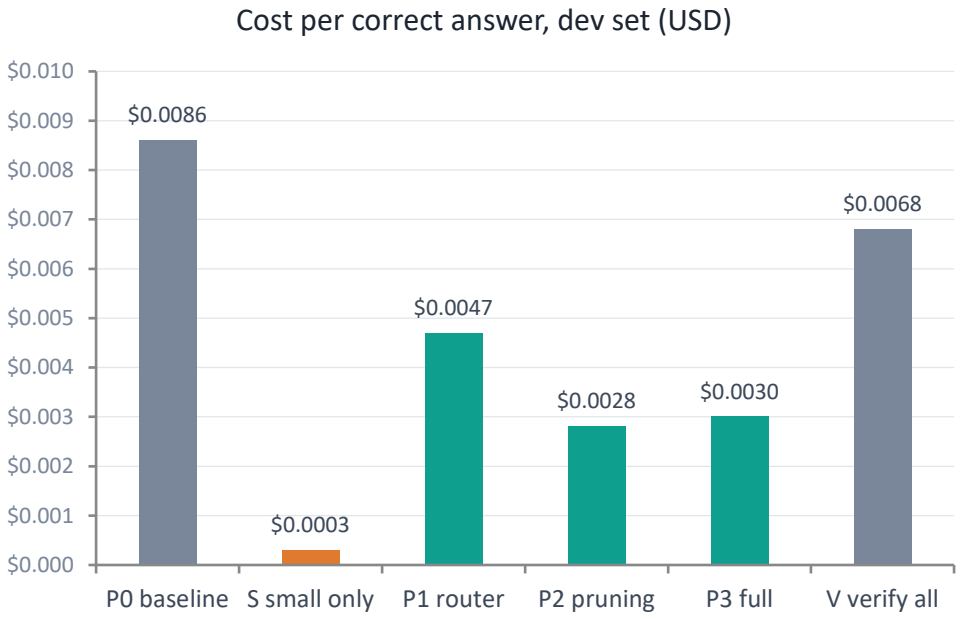
- accuracy ≥ baseline − 2 pp
- p95 latency ≤ 1.25 × baseline
- headline metric: cost per successful task (failed and verified attempts included)

Held-out protocol

- 150 tasks, 6 question types never used in development
- secret seed, file generated outside the repository, never opened
- one run per policy after all thresholds were frozen

109 unit/component tests · pruner recall measured (100 % of tasks get every table their gold SQL needs, 2.6 of 12 sent) · mock end-to-end run in CI on every push · 21 logged decisions

Results: dev ladder and held-out set



policy	accuracy	p95	Δ cost
P0	100.0 %	3.4 s	—
S	84.7 %	7.0 s	−96 %
P1	98.0 %	3.5 s	−46 %
P2	99.3 %	3.0 s	−68 %
P3	100.0 %	4.3 s	−65 %
V	100.0 %	7.1 s	−21 %

−68 % cost per correct answer on 150 unseen tasks: $0.0093 → $0.0030, accuracy 99.3 % → 98.7 %, p95 3.1 s → 4.4 s

Robustness (P3, no retuning, 60 tasks each): renamed schema 100 % · Arabic questions 100 % · small model swapped to qwen3.7-flash 100 % but p95 29 s, which the controller exposes. Floors met by P1 and P2; P3 misses the p95 floor by 0.1 s (dev) / 0.5 s (held-out) and is reported as such.

Why this matters beyond the benchmark

1 Compute follows risk

Spend where the outcome is at risk, not uniformly. The same pattern fits any task with a cheap first-stage check: SQL execution, unit tests, schema validation.

2 Measured, not asserted

Each component earns its own row; the naive extremes (always cheap, always verify) are shown to lose on the same tasks.

3 Found by measurement, fixed early

Renamed-schema pruner recall 46.7 % → 100 %; Arabic 92.2 % → 100 %; a verifier that spent 85 % of its budget checking the large model against itself, removed.

4 Honest accounting

Failed, repaired, verified and escalated attempts are all in the numerator. The one floor the full system misses is printed, not hidden.

Limitations and what comes next

Limitations

- Synthetic data and templated questions (paraphrased, not human-written)
- One database domain; a lexical router
- p95 floor (1.25 × one flagship call) missed by the full system by 0.1–0.5 s; pruning alone meets it
- Model ids and prices are a 30 Oct 2026 snapshot of OpenRouter
- Two days, one person

Next

- A learned router trained on the logged traces
- Prompt caching for the shared prefix; a third tier
- Verifier calibration: false-accept rate against gold
- Multi-turn sessions and real customer schemas
- Latency-aware routing when a provider slows down

68 % less compute per correct answer, inside the accuracy floor

On 150 unseen tasks, with the evidence to prove it: a measured baseline, one row per component, robustness under drift, and a held-out run nobody looked at.

Repo and one-command reproduction: github.com/Ahmednsh/humain-less-compute

Results: results/report.html · Decisions: DECISIONS.md · Demo video: see project page